

Course: AI Champions Programme

GitHub Copilot Enterprise, Spec-Driven Development, Agentic Embedded Engineering

NIIT StackRoute Programme Design | Honeywell Embedded Engineering Context

Program Introduction

The programme follows an incremental engineering journey rather than a collection of AI tool topics. Participants first establish the enterprise context and AI Champion role for embedded software engineering, understand current delivery pain points and success measures, then build practical GitHub Copilot capability, recap prompt engineering and move into context engineering, learn SDD, practice a complete greenfield embedded software SDLC, learn brownfield embedded software evolution and best practices, validate SDD-developed features through a proper software-level test strategy, extend agents with MCP, enable business and program roles through studio-style workflows in Atlassian Rovo Studio, automate PR quality with LLM-as-Judge, observe AI workflows with Agent Prism, measure token economics and ROI, and finally demonstrate the complete workflow through the Honeywell Embedded Engineering Capstone and Adoption Roadmap.

GitHub Copilot Enterprise is used as the primary AI engineering platform. Participants first build a practical understanding of Copilot capabilities including chat, inline code assistance, code explanation, refactoring, debugging, test generation, agent mode, repository-aware assistance, model/task selection, GitHub.com workflows, terminal/CLI workflows, PR/code-review assistance, and reusable instructions/skills. The technical work is grounded in C/C++ embedded software and firmware repositories, build systems, interfaces, configuration, diagnostics, state machines, drivers/adapters, and existing engineering conventions. The programme then develops two complementary engineering flavours: greenfield embedded software built from specifications and brownfield firmware enhancement performed safely within an existing codebase.

A core differentiator is the direct comparison between prompt-only development and specification-led embedded engineering, followed by disciplined validation. Module 3 deliberately recaps practical prompt engineering before moving into context engineering for embedded repositories: source files, headers, interfaces, build/configuration files, hardware abstraction layers, design documents, coding standards, issue history and tests. Participants then use SDD for both greenfield and brownfield work, validate features with an embedded-appropriate test strategy, and measure implementation time, quality, regression risk, review effort, context usage, token consumption and AI cost. MCP is treated as a major capability for connecting agents to approved engineering tools and workflows. Atlassian Rovo Studio is then introduced as the studio-style layer through which program and product roles author governed workflows on top of that engineering foundation.

Agent Prism is positioned as the live practice platform for monitoring and observability. Participants learn why embedded engineering teams need visibility into agent behaviour, traces, failures, token usage, cost leakage, quality signals, control and operational adoption. Agent Prism is introduced early as the monitoring lens, then used later to connect AI-assisted engineering activity with measurable quality, cost and adoption outcomes.

Business Problems This Programme Solves

- Slow and inconsistent embedded software implementation cycles caused by repetitive code analysis, firmware changes, test creation, defect triage, documentation, and review activities.
- Risk of AI-generated C/C++ changes introducing defects, violating established firmware architecture or coding standards, breaking interfaces/state behaviour, or creating difficult-to-diagnose regressions in a brownfield codebase.
- Weak translation of embedded requirements, constraints and acceptance criteria into precise implementation tasks, especially when prompt-only development is used without a specification and traceability discipline.
- Manual and inconsistent validation and PR review, making it difficult to prove that SDD-developed changes satisfy requirements, preserve existing behaviour, and meet embedded quality and traceability expectations.
- Excessive token consumption and poor context discipline when large embedded repositories, generated headers, configuration files, design documents and test artefacts are supplied without task-scoped context.

- Limited visibility into AI usage, quality, failure patterns, token economics and measurable productivity/ROI from AI-assisted embedded engineering.
- Program, product and other non-code roles have no governed route from business intent to agentic execution, so program reporting, issue triage and release coordination either stay manual or are pushed onto engineering teams, and any workflows those roles do build sit outside review, permission and ownership controls.

Indicative Design and Content Coverage

Pre-requisites

- Participants should have working familiarity with embedded software/firmware engineering, SDLC/PDLC practices, Git/GitHub collaboration, and the development lifecycle used for C/C++ embedded projects.
- Technical participants should be comfortable with C and/or C++, source/header organisation, build systems/toolchains, debugging, unit testing, branching, pull requests, code review, and basic embedded concepts such as hardware abstraction, interfaces, state machines, drivers/adapters, configuration, and resource constraints.
- Honeywell should provide representative firmware/software architecture information, coding standards, build/test conventions, and a sample or sanitized C/C++ codebase. Hardware integration will not be demonstrated; the live programme focuses on embedding software development and software-level validation.
- Recommended pre-read/self-paced foundation: AI fundamentals bootcamp, intro to GenAI, intro to agents, and responsible AI awareness as listed in the Honeywell training sheet.
- Access should be arranged for GitHub Copilot Enterprise, approved embedded repositories, build/test projects or sample codebases, required C/C++ toolchains and static-analysis/test tooling, approved MCP tools, Atlassian Rovo Studio access on a Jira/Confluence tenant or sandbox with a representative project and space, and Agent Prism sandbox/demo access.

Course: AI Champions Programme for Embedded Engineering Teams - NIIT Delivery Design

Total Duration: 24 Hours — 3 Full Days (8 hours/day), delivered as an intensive hands-on programme with logical progression from enterprise context and Copilot foundations through SDD, greenfield and brownfield embedded engineering, testing, MCP, studio-style workflow authoring in Atlassian Rovo Studio, PR quality, observability, economics, and the final capstone.

Target Roles

- Common foundation for all: Architects, Developers, Testers, DevOps, SRE, Engineering Leads, and selected Program/Product roles
- Technical deep-dive emphasis: Embedded/Firmware Developers, Architects, Testers, DevOps, SRE, and Engineering Leads

Key Outcomes

- Apply an incremental AI-assisted embedded software lifecycle covering enterprise context, GitHub Copilot capabilities, prompt and context engineering, specification, planning, implementation, embedded testing, PR review, MCP-enabled tool integration, studio-style workflow authoring, observability, economics and adoption.
- Use GitHub Copilot Enterprise effectively across major engineering workflows including chat/conversation, inline code assistance, code explanation, refactoring, debugging, test generation, agent mode, repository-aware assistance, model/task selection, GitHub.com workflows, terminal/CLI workflows, PR/code-review assistance, and reusable engineering instructions/skills.
- Author structured embedded software specifications using an SDD workflow — e.g., GitHub Spec Kit or an OpenSpec-equivalent pattern — and use requirements, constraints, interfaces, acceptance criteria, plan and tasks to drive traceable implementation.
- Build greenfield embedded software from specification through architecture, implementation, test strategy, build/validation and PR/release readiness, while understanding the boundaries between software development and hardware integration.
- Enhance brownfield C/C++ firmware safely by understanding existing architecture, interfaces, build/configuration dependencies, coding standards, state behaviour, extension points, design patterns and regression risks before modifying code.
- Design and execute a proper embedded software test strategy for SDD-developed features, including host-based unit tests, mocks/stubs/fakes, integration testing, static analysis, build validation, boundary/negative testing and E2E/software-level validation. Hardware-in-the-loop concepts may be discussed but are not demonstrated.
- Build MCP-enabled engineering workflows that connect agents to approved repositories, build/test tools, issue systems, documentation, CI services and other engineering utilities; understand tool boundaries, permissions, approvals, failure handling and governance.
- Implement AI-assisted PR quality gates including specification compliance, C/C++ coding standards, design constraints, test evidence, regression risk, security/safety considerations and LLM-as-Judge patterns with human escalation.
- Create reusable embedded engineering skills such as design explainers, test enhancement assistants, defect/RCA assistants, PR reviewers and specification validators, then use Agent Prism to monitor traces, quality signals, failures, token usage, cost patterns, adoption and ROI.
- Enable program, product and other semi-technical roles to author studio-style workflows in Atlassian Rovo Studio — grounding agents on Jira and Confluence content, defining triggers, actions, approval flows and human-in-the-loop checkpoints — and hand those workflows to technical owners for implementation without bypassing engineering governance.

Detailed Design

Module Name	Sub-topics / Detailed Technical Coverage	Duration (Hours)	Hands-on	Tools Needs
1. AI Champions Kick-off and Embedded Engineering Enterprise Context	Programme kick-off, why Honeywell-style embedded engineering teams need AI champions, shift from isolated AI experimentation to disciplined enterprise adoption, role-based expectations across embedded developers, architects, testers, DevOps, SRE, engineering leads and selected program/product roles, review of the Honeywell training-sheet themes, business and engineering risks of unmanaged AI/agent usage, embedded software lifecycle context, and introduction to Agent Prism as the monitoring and measurement lens.	1.5	Workshop: identify current pain points in embedded software/firmware delivery, feature implementation, defect resolution, build/test cycles, PR review latency, legacy-code challenges and AI usage blind spots. Map current-state versus target-state engineering workflow, identify representative greenfield and brownfield use cases, and define success metrics.	Slides, role-mapping canvas, current/target-state workflow canvas, success-metrics worksheet, Agent Prism overview
2. GitHub Copilot Enterprise Features for Embedded Engineering	GitHub Copilot Enterprise capabilities including chat/conversation, inline code assistance, code explanation, refactoring, debugging, test generation, agent mode, repository-aware assistance, model/task selection, GitHub.com workflows, terminal/CLI workflows, PR/code-review assistance, custom/reusable instructions and skills, and safe enterprise usage. Embedded focus: C/C++ source/header analysis, interfaces, configuration, diagnostics, state machines, drivers/adapters and build-related code.	2	Hands-on: use different Copilot capabilities on a sample C/C++ repository to generate, explain, refactor and review code; debug a defect; generate tests; use agent mode for a repository task; compare manual versus AI-assisted execution.	GitHub Copilot Enterprise, IDE, GitHub.com, terminal/CLI, sample C/C++ embedded codebase
3. Prompt Engineering Recap and Context Engineering	Recap practical prompt engineering for embedded software tasks: task framing, instructions, constraints, examples, decomposition, iterative prompting and verification. Move into context engineering for embedded repositories: source/header relationships, interfaces/contracts, build files, configuration, hardware-abstraction boundaries, design notes, coding standards, issue history and tests; context selection, layering, compression, task-scoped context, context boundaries, token-aware practices and avoiding context inflation.	2	Hands-on: solve an embedded engineering task using prompt-only context, then progressively improve it using structured repository and design context. Compare output quality, token usage, iteration count and completion time.	Copilot, prompt/context templates, C/C++ repository examples, token usage worksheet

Module Name	Sub-topics / Detailed Technical Coverage	Duration (Hours)	Hands-on	Tools Needs
4. Spec-Driven Development for Embedded Software	SDD fundamentals for embedded engineering, requirements, functional/non-functional constraints where applicable, interfaces/contracts, acceptance criteria, specification quality, spec → plan → tasks → implementation, GitHub Spec Kit/OpenSpec patterns, traceability, change control, and prompt-only versus spec-driven development.	2	Hands-on: create a specification for an embedded software feature, define interfaces and acceptance criteria, generate plan/tasks, implement against the specification, and validate traceability.	GitHub Copilot, GitHub Spec Kit / OpenSpec, specification templates, acceptance-criteria checklist
5. Embedded Development – SDD to Complete Software SDLC	Problem framing, architecture, interface design, module structure, C/C++ implementation, testing strategy, build validation, static analysis, CI/CD, release readiness. Repository archaeology, architecture analysis, dependency mapping, interface contracts, HAL boundaries, change impact analysis, extension points, minimal-change strategy, backward compatibility, regression protection and safe evolution of existing systems.	3.5	Hands-on: Build a greenfield feature from specification through implementation and testing. Analyze a brownfield firmware repository, create a change plan, implement enhancements safely, validate behavior preservation and compare greenfield vs brownfield approaches.	GitHub Copilot Enterprise, Spec Kit/OpenSpec, C/C++ Toolchain, Build System, CI Pipeline, Static Analysis Tools, Test Frameworks, Brownfield Repository
6. Embedded Test Strategy, Validation and Regression Engineering	Test strategy derived from SDD, specification-to-test traceability, host-based unit testing, mocks/stubs/fakes, integration testing, API/interface validation, boundary and negative testing, static analysis, compiler/build validation, sanitizers where applicable, regression testing, software-level E2E validation, test evidence and acceptance-criteria validation. Hardware-in-the-loop and physical hardware integration are discussed only as concepts, not demonstrated.	2.5	Hands-on: derive a test strategy from the SDD feature, create/update unit and integration tests, use mocks/stubs/fakes for hardware-dependent interfaces, run build/static checks, execute software-level integration/E2E tests, inject a deliberate defect, and prove the strategy catches it.	C/C++ test framework, compiler/toolchain, static-analysis tools, mocks/stubs/fakes, sanitizers where applicable, CI pipeline

Module Name	Sub-topics / Detailed Technical Coverage	Duration (Hours)	Hands-on	Tools Needs
7. MCP-Enabled Agentic Embedded Engineering and Reusable Skills	MCP architecture, servers/tools/resources, connecting agents to approved embedded repositories, build/test tools, CI services, issue systems, documentation and engineering utilities; tool selection, permissions, approvals, context/state boundaries, failure handling, security and governance. Multi-agent patterns are introduced through practical engineering roles such as planner, architect, developer, tester, reviewer and release checker. Package reusable embedded skills, instructions, prompt packs, test/review heuristics and agent behaviours with versioning and ownership.	2	Hands-on: build an MCP-enabled workflow such as repository analysis → specification validation → build/test execution → result analysis or PR evidence collection. Then package a reusable embedded skill such as design explainer, test enhancement assistant, PR reviewer or RCA assistant.	GitHub Copilot/ agent environment, MCP server examples, approved engineering tools/ APIs, GitHub assets, skill templates
8. Studio-Style Workflows for Business and Engineering Collaboration	How non-code or semi-technical users shape agent behaviour through studio-style workflows in Atlassian Rojo Studio: agent authoring for program and product roles, grounding agents on Jira issues, Confluence design and release documentation and other connected engineering sources, translating business intent into agentic execution, triggers and automation rules, actions and connectors, approval flows, human-in-the-loop checkpoints, permission-aware access to project data, and versioning and ownership of published workflows. Covers how business users collaborate with technical owners without bypassing governance, and where the boundary sits between a studio workflow and an engineering agent. Positioned as the enterprise workflow-design layer alongside the Copilot and MCP engineering workflows built earlier.	2	Hands-on: program and product users define an agent-assisted workflow in Atlassian Rojo Studio for program reporting, issue triage, or release coordination. Ground it on a sample Jira project and Confluence space, configure the trigger, actions and approval steps, test against representative project data, review permission and governance implications, and hand it over to technical owners for implementation.	Atlassian Rojo Studio, Jira/ Confluence sandbox tenant, sample project and space, workflow canvas, studio-pattern examples, business-user templates

Module Name	Sub-topics / Detailed Technical Coverage	Duration (Hours)	Hands-on	Tools Needs
9. PR Process Automation, Quality Gates and LLM-as-Judge	AI-assisted PR review for C/C++ embedded projects, specification compliance, coding/design standards, test evidence, compiler/build results, regression risk, security and safety considerations, review heuristics, evidence-based comments, LLM-as-Judge evaluation rubrics, confidence, human escalation and review-cycle reduction.	1.5	Hands-on: configure a PR quality flow, run AI review and LLM-as-Judge checks against a deliberately flawed embedded change, capture evidence from tests/build/static analysis, compare review time and quality, and close the loop with human approval.	GitHub repo, PR templates, C/C++ quality checklist, CI pipeline, LLM-as-Judge rubric
10. Agent Prism for Monitoring, Observability and Enterprise Control	Why monitoring agentic engineering matters, observability concepts, traces, replay, failure patterns, prompt/output drift, quality/control signals, token usage, cost-leak visibility, thresholds and alerts, adoption tracking, operational review, and connecting AI telemetry to embedded engineering outcomes.	2	Hands-on: ingest or simulate engineering-agent run data in Agent Prism, inspect traces and failures, identify token/cost leakage, compare successful and poor-quality runs, and relate findings to implementation, test and PR quality KPIs.	Agent Prism sandbox/demo, sample run telemetry, monitoring scorecards
11. ROI, Token Economics and Deployment Governance	Engineering productivity versus production impact, cost per accepted production-quality output, token governance, saved implementation/review/testing effort, quality and regression signals, business-value measurement, when an AI workflow should be scaled/tuned/restricted/retired, governance and leadership reporting for embedded engineering.	1	Hands-on: calculate before/after metrics for an embedded engineering workflow, estimate AI cost per accepted production-quality output, build an ROI model using saved effort and reduced review/test time, and define governance actions.	ROI worksheet, token economics template, Agent Prism metrics, KPI dashboard
12. Honeywell Embedded Engineering Capstone and Adoption Roadmap	Teams integrate the complete embedded engineering journey into one realistic adoption plan: GitHub Copilot Enterprise workflow, embedded SDD, greenfield or brownfield engineering approach, proper software-level test strategy, reusable embedded skill pack, MCP-enabled workflow, PR quality automation, LLM-as-Judge, context/token discipline, Agent Prism monitoring model, business case, governance controls and enterprise rollout roadmap. Hardware integration is not demonstrated.	2	Capstone: present one embedded software use case with solution blueprint, specification/plan, implementation approach, test strategy and evidence, MCP workflow, PR/LLM-Judge evidence, Agent Prism monitoring design, ROI model and 90-day adoption roadmap.	Greenfield starter or brownfield C/C++ repository, GitHub repo/PR pipeline, Spec Kit/ OpenSpec, C/C++ toolchain/test tools, MCP tools, Agent Prism, capstone canvas, evaluation rubric

Coverage Alignment to the Uploaded Honeywell Embedded Engineering Training Sheet

- The programme is deliberately structured as a 12-module progression so each concept builds on the previous one: embedded enterprise context and AI Champion role → GitHub Copilot capabilities → prompt engineering recap and context engineering → SDD → greenfield and brownfield embedded software SDLC → embedded test strategy and validation → MCP and reusable skills → studio-style workflows in Atlassian Rovo Studio → PR quality/LLM-as-Judge → Agent Prism → ROI/token economics/governance → Honeywell Embedded Engineering Capstone and Adoption Roadmap.
- Discovery and Launch themes are reflected through embedded engineering use-case identification, design thinking, requirements framing, repository/context understanding, prompt/context engineering, and translating firmware/software requirements and constraints into implementation-ready specifications.
- Development and Test themes are strengthened through GitHub Copilot agent mode, SDD, C/C++ embedded development, build/toolchain awareness, brownfield codebase evolution, host-based testing, mocks/stubs/fakes, static analysis, regression validation, PR automation, code review, MCP/tool integration, reusable engineering skills and AI usage cost optimisation.
- Agentic and COTS AI themes are addressed through MCP server/tool integration, practical multi-agent engineering patterns, and studio-style workflow authoring in Atlassian Rovo Studio for business and program roles.
- Agent Prism remains the practical enterprise monitoring and observability layer so teams can connect AI-assisted embedded engineering activity with traces, failures, quality, token usage, cost, adoption and ROI.

Learning Pedagogy

- Instructor-led sessions use realistic, embedded engineering scenarios and C/C++ repositories. Participants learn the enterprise context first, then build Copilot capability before moving into prompt/context engineering, SDD, greenfield development, brownfield evolution, testing, MCP, studio-style workflow authoring, PR quality, observability and economics.
- Guided labs follow a complete embedded software lifecycle: understand/frame the engineering problem, inspect repository and architecture context, specify, plan, implement, compile/build, test, review, integrate approved tools through MCP, observe AI behaviour, measure economics, and demonstrate adoption. Hardware integration is intentionally excluded from the hands-on flow.
- Participants recap prompt engineering and then compare prompt-only implementation with structured embedded context and SDD workflows, evaluating differences in implementation time, output quality, context usage, testing effort, review effort, specification compliance, token usage and cost.
- Design-thinking activities focus on embedded constraints, interfaces, failure modes, existing architecture, software/hardware boundaries, acceptance criteria and change impact before AI is asked to implement a solution.
- Testing clinics connect AI productivity directly to engineering confidence: test strategy from the specification, host-based unit testing, mocks/stubs/fakes, integration testing, build/static checks, regression validation and software-level E2E testing.
- MCP is taught as a practical mechanism for extending AI-assisted embedded engineering with governed tools and engineering services. Participants design and use MCP-enabled workflows for repository analysis, build/test execution, documentation, issue systems and PR evidence.
- Studio-style workflow sessions give program and product roles a governed way to participate. Using Atlassian Rovo Studio, participants ground agents on Jira and Confluence content, define triggers, actions, approval flows and human-in-the-loop checkpoints, test against representative project data, and hand

completed workflows to technical owners so business intent reaches execution without bypassing engineering governance.

- Agent Prism labs connect traces, token usage, cost, quality and failure patterns to the embedded engineering success metrics. The ROI/governance module then converts these observations into measurable engineering outcomes before the final capstone.

Free Value-Adds / Client Handholding

- 30-day catch-up review session at no additional cost to review what participants have applied, what has been piloted in embedded software workflows, and where adoption is blocked.
- One complimentary masterclass after the 30-day review focused on advanced lessons learned, common implementation mistakes, SDD maturity, embedded testing strategies, MCP workflow patterns, studio workflow patterns for business roles, and next-step AI engineering maturity.
- Post-training assessment framework to review feature implementation cycle time, PR review time, regression/defect signals, reusable skill/spec adoption, test effectiveness, token spend discipline, and early ROI signals from embedded engineering pilots.
- Short gap-closure recommendations based on participant progress, leadership expectations, and identified blockers in governance, tooling, brownfield adoption, testing, or workflow design.

Delivery Recommendations

- Recommended audience: Honeywell embedded engineering teams with role-based participation from firmware/software developers, architects, testers, DevOps, SRE, engineering leads, and selected program/product roles.
- Preferred delivery model: 3 full days with a progressive mix of instructor-led concepts, guided demonstrations, design-thinking/use-case workshops, hands-on C/C++ engineering labs, testing/validation exercises, MCP, Atlassian Rovo Studio and Agent Prism practice, and a final capstone/adoption exercise.
- Primary tool/platform: GitHub Copilot Enterprise for technical users, GitHub Spec Kit/OpenSpec for SDD, C/C++ toolchains and software-level testing/static-analysis tools, MCP servers/tools for governed engineering integrations, Atlassian Rovo Studio for studio-style workflow authoring by business and program roles, and Agent Prism for monitoring, observability, token economics and ROI.
- Recommended batch design: common foundation followed by two targeted engineering journeys — greenfield SDD/complete embedded software SDLC and brownfield firmware enhancement/safe evolution — with Copilot capabilities and prompt/context engineering established first, and MCP integrated after the core engineering workflow is understood, followed by studio-style workflow authoring for the program and product participants. RAG is intentionally excluded.
- Outcome expectation: every batch should leave with a working greenfield embedded SDD workflow, a working brownfield feature-change approach, a documented embedded test strategy with software-level validation evidence, reusable engineering assets, MCP-enabled workflow patterns, a governed Atlassian Rovo Studio workflow authored by a business or program role, PR quality automation, Agent Prism observability evidence, a token/cost and ROI model, and a completed Honeywell Embedded Engineering Capstone and Adoption Roadmap.
- Client enablement needed: approved GitHub access, representative C/C++ firmware/software repositories or sanitized samples, firmware/software architecture documentation, coding standards, build/toolchain setup, existing tests, static-analysis and test tooling, approved MCP servers/tools, Atlassian Rovo Studio access with a representative Jira project and Confluence space, PR/CI pipelines or mock pipelines, reusable standards/checklists, and Agent Prism practice environment or demo dataset. Hardware integration is not required for the training labs.